Lab2: Implementing a REST server in Node.JS with Express.JS

Directory Structure
lab2/

- ➔ index.js ← Express, port:3000
- ➔ scratch.js ← Scratch, port:3001
- ➔ data.json
- ➔ package.json
- ➔ controllers/
  - ◆ MainController.js
  - ◆ StudentController.js
  - ◆ ProfController.js
  - ◆ ModuleController.js

LAB:

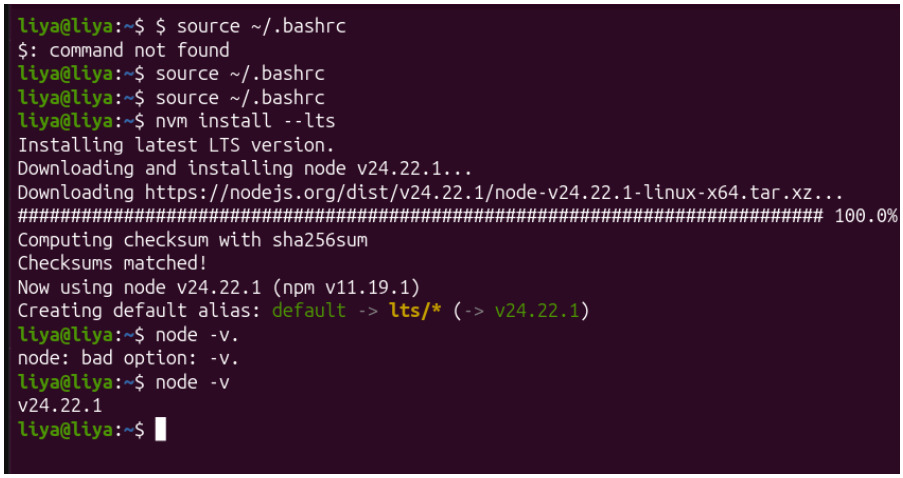
1. Open a terminal;
2. Run the following command: `curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash`
3. Export, if not automatically done, the following environment variable into your .bashrc file:

   ```
   export NVM_DIR="$HOME/.nvm"
   [ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"  # This loads nvm
   [ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion"  # This loads nvm bash_completion
   ```

4. Open a new terminal or source the .bashrc file:

   ```
   $ source ~/.bashrc
   ```

5. Install the latest Long Term Support (LTS) version of Node.JS;
6. Check the version of Node.JS using `node -v`.



Now that Node.JS is installed, you can create a new project:

1. Create a directory lab2/ into ngs/ and open it with your terminal ;
2. Execute the `npm init` command and let the wizard guide you. Leave the default values except for the type, and optionally the name. We will use ECMAScript rather than CommonJS ;
3. By default, the name of the package corresponds to the name of your directory. You can also change it if you want.
4. Create an index.js file at the root of the repository and put the following content: `console.log('Hello world');`
5. Execute the file with `node index.js`

Right now, you should see `Hello world` displayed in the terminal and the script stops its execution.

```
liya@liya:~/ngs/lab2$ npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help init` for definitive documentation on these fields and exactly what they do.

Use `npm install <pkg>` afterwards to install a package and save it as a dependency in the package.json

Press ^C at any time to quit.
package name: (lab2)
version: (1.0.0)
description:
entry point: (index.js)
test command:
git repository:
keywords:
author:
license: (ISC)
type: (commonjs) module
About to write to /home/liya/ngs/lab2/package.json:

{
  "name": "lab2",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC",
  "type": "module"
}

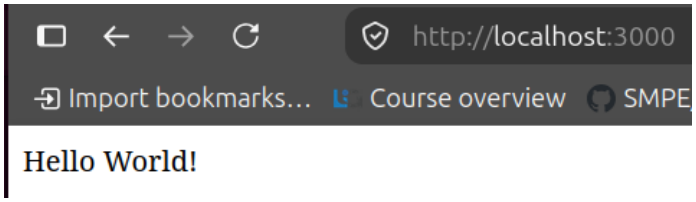

Is this OK? (yes)
liya@liya:~/ngs/lab2$ echo "console.log('Hello world');" > index.js
liya@liya:~/ngs/lab2$ node index.js
Hello world
liya@liya:~/ngs/lab2$
```

Installing the dependency

1. Install the Express.js dependency using `npm i --save express`
2. In the index.js file you created earlier, replace its content with the following code adapted from the example available at the Express homepage:

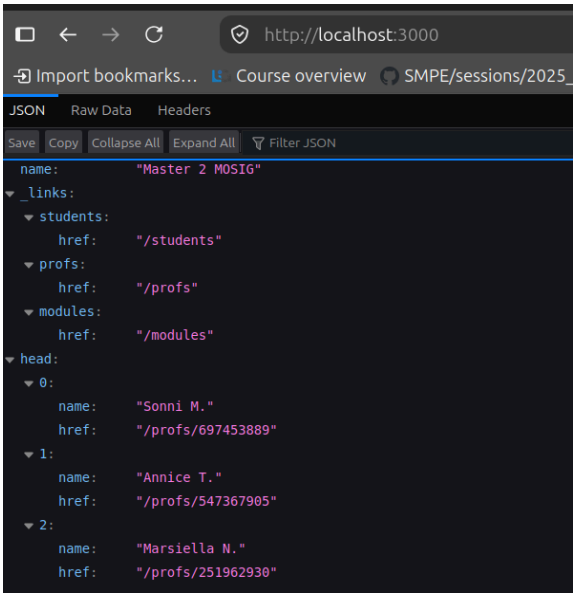
```
import express from 'express';
const app = express()
const port = 3000
app.get('/', (req, res) => {
  res.status(200).send('Hello World!')
})
app.listen(port, () => {
  console.log(`Example app listening on port ${port}`)
})
```

3. Execute the script with `node index.js` and open http://localhost:3000.



Implementing your first route

Download the data.json file into the same directory that index.js ;
Import the `fs` built-in package (you do not have to install it) into index.js and import the JSON data using the `fs.readFileSync()` function. You may want to have a look at the fs documentation to know how to use this function ;
Change the code into `app.get('/', (req, res) => { /* ... */ });` **such that it returns an incomplete representation of the Mosig department** as it is done in the REST server available during lab1.Handle error case if the file is not found.



At this point, your first route is supposed to work, but you can refactor a bit the code, so that the controllers are not stored in a single `index.js` file.

Refactor the code such that the controllers will be located in a separate directory `controllers`. Right now, you are supposed to have only one controller, e.g., `MainController`, but you will have to create others for students, professors, etc.

index.js

```
import express from 'express';
import { getMain } from './controllers/MainController.js';
const app = express();
const port = 3000;

app.get('/', getMain);
app.listen(port, () => {
  console.log(`Example app listening on port ${port}`);
});
```

MainController.js

```
import fs from 'node:fs';
export function getMain(req, res) {
  try {
    const data = JSON.parse(fs.readFileSync('./data.json', 'utf-8'));
    const result = {
      name: data.name,
      _links: {
        students: { href: '/students' },
        profs: { href: '/profs' },
        modules: { href: '/modules' }
      },
      head: data.responsibles.map(num => {
        const prof = data.profs.find(p => p.number === num);
        return {
          name: `${prof.lastname} ${prof.firstname[0]}.`,
          href: `/profs/${num}`
        };
      })
    };
    res.status(200).json(result);
  } catch (err) {
    res.status(500).json({ error: 'Data file not found' });
  }
}
```

Handling route parameters

Dynamic routes allows you for specifying routes that accept parameters, such as an id, a student's ine, or a prof's professional number.

- Read the documentation about routing with Express.js and find how to specify route parameters.
- Implement it to specify a single (dynamic) route for students, modules, etc.
- Use the `data.json` file to fill the data. Remember that you have to handle errors.

StudentController.js × ModuleController.js

```
import fs from 'node:fs';
export function getStudent(req, res) {
  try {
    const data = JSON.parse(fs.readFileSync('./data.json', 'utf-8'));
    const student = data.students.find(s => s.ine === req.params.ine);

    if (!student) {
      return res.status(404).json({ error: 'Student not found' });
    }
    res.status(200).json(student);
  } catch (err) {
    res.status(500).json({ error: 'Data file not found' });
  }
}
```

ProfController.js × StudentController.js

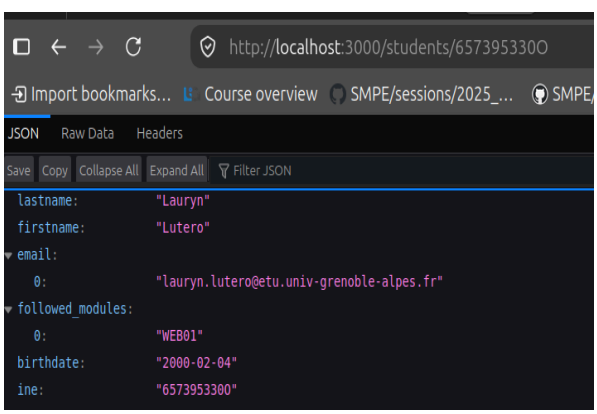
```
import fs from 'node:fs';
export function getProf(req, res) {
  try {
    const data = JSON.parse(fs.readFileSync('./data.json', 'utf-8'));
    const prof = data.profs.find(p => p.number === req.params.number)

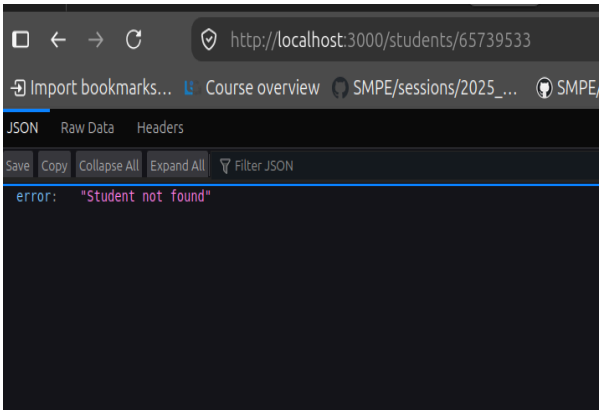
    if (!prof) {
      return res.status(404).json({ error: 'Professor not found' });
    }
    res.status(200).json(prof);
  } catch (err) {
    res.status(500).json({ error: 'Data file not found' });
  }
}
```

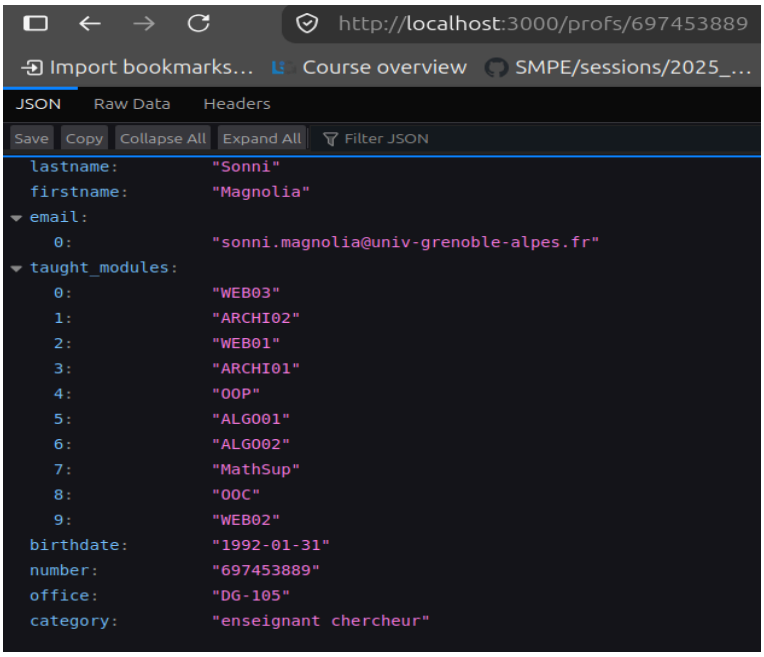
ModuleController.js × ProfController.js

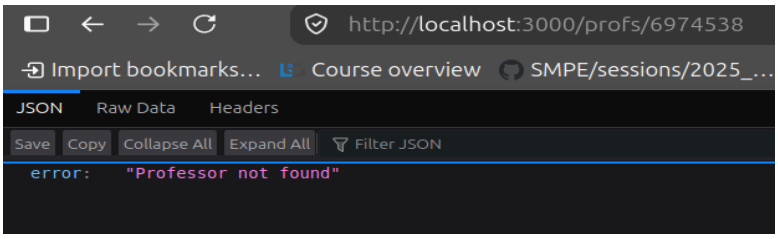
```
import fs from 'node:fs';
export function getModule(req, res) {
  try {
    const data = JSON.parse(fs.readFileSync('./data.json', 'utf-8'));
    const id = req.params.id.toLowerCase();
    const mod = data.modules.find(m => m.code_module.toLowerCase() ===

    if (!mod) {
      return res.status(404).json({ error: 'Module not found' });
    }
    res.status(200).json(mod);
  } catch (err) {
    res.status(500).json({ error: 'Data file not found' });
  }
}
```









Handling data in the payload

Use `body-parser` to implement `post`, `put`, etc. routes.

POST

```
liya@liya:~/ngs/lab2$ curl -i -X POST -H "Content-Type: application/json" -d '{"ine":"TEST000
001","lastname":"Test","firstname":"User","email":["t@u.fr"],"followed_modules":[],"birthdate
":"2000-01-01"}' localhost:3000/students
HTTP/1.1 201 Created
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 123
ETag: W/"7b-QFkp9WMzUmqPmAqEHOQ7Bs8Gv1E"
Date: Sun, 25 Oct 2026 15:27:26 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"ine":"TEST000001","lastname":"Test","firstname":"User","email":["t@u.fr"],"followed_modules
```

GET

```
liya@liya:~/ngs/lab2$ curl -i localhost:3000/students/TEST000001
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 123
ETag: W/"7b-QFkp9WMzUmqPmAqEHOQ7Bs8Gv1E"
Date: Sun, 25 Oct 2026 15:27:55 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"ine":"TEST000001","lastname":"Test","firstname":"User","email":["t@u.fr"],"followed_modules
```

PUT

```
liya@liya:~/ngs/lab2$ curl -i -X PUcurl -i -X PUT -H "Content-Type: application/json" -d '{"lastname":"Changed"}' localhost
:3000/students/TEST000001
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 126
ETag: W/"7e-egHbD/CmTRRny4EBbZgqaYNu8ro"
Date: Sun, 25 Oct 2026 15:28:02 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

DELETE

```
:liya@liya:~/ngs/lab2$ curl -i -X DELETE localhost:3000/students/TE
ST000001lab2$ curl -i -X DELETE localhost:3000/students/TEST000001
HTTP/1.1 204 No Content
X-Powered-By: Express
Date: Sun, 25 Oct 2026 15:28:11 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

404 Not Found

```
liya@liya:~/ngs/lab2$ curl -i localhost:3000/students/TEST000001
HTTP/1.1 404 Not Found
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 29
ETag: W/"1d-/w9leX+CSVwScG3mclc8pAqpxGg"
Date: Sun, 25 Oct 2026 15:28:17 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Implementing a REST server from scratch

File: scratch.js

The Express version occupies port 3000, so the scratch version uses port 3001. Both servers can run simultaneously.

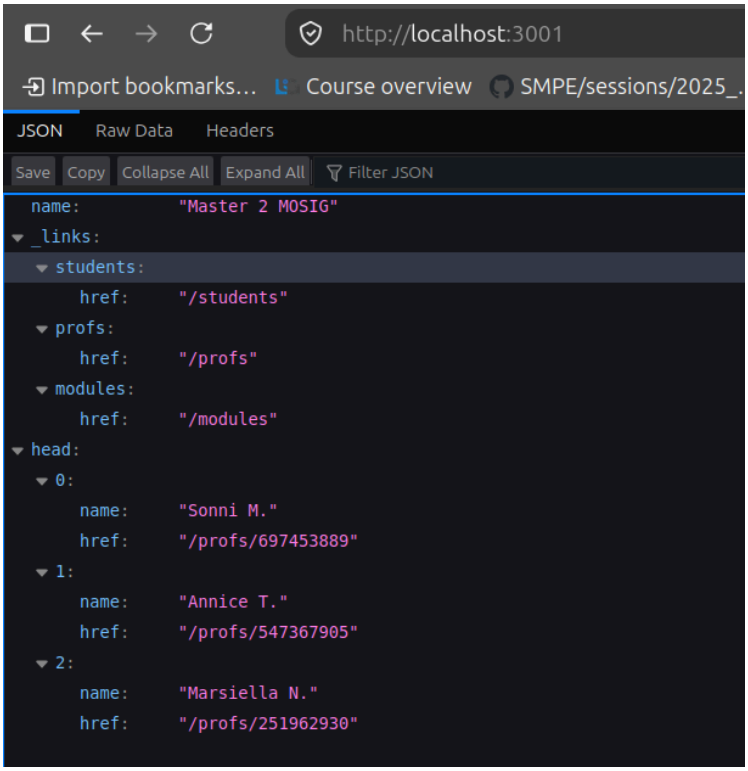
Routing table: Routes are stored in a plain array. Each entry contains four fields: method, regex, keys and handler.

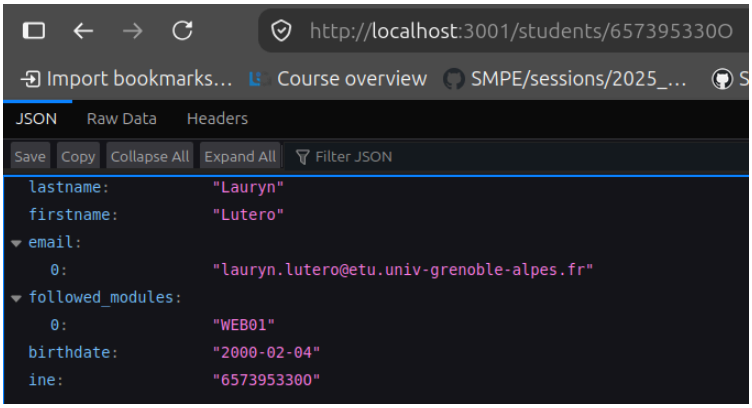
Routes are registered with addRoute(method, path, handler).

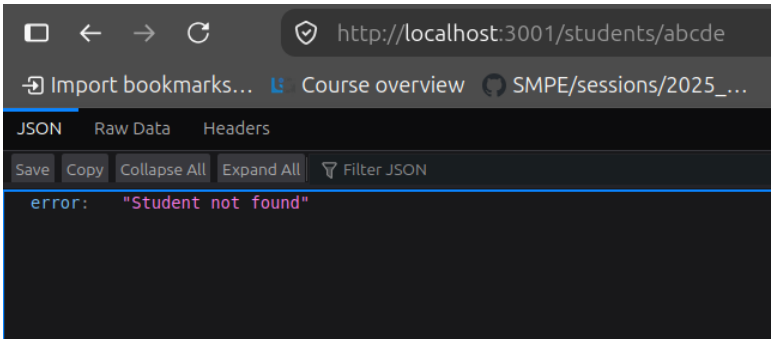
When a request comes in, the server goes through the array and uses the first route that has the same method and a matching path. If nothing matches, it sends a 404.

I did not rewrite the controllers. The scratch server uses the same ones as the Express version. They call res.status(...).json(...), which plain Node.js does not have, so I added three small helpers on res: status, json and send.

Result examples:




http://localhost:3001/students/6573953300
Import bookmarks...
Course overview
SMPE/sessions/2025_...
JSON
Raw Data
Headers
Save
Copy
Collapse All
Expand All
Filter JSON
lastname: "Lauryn"
firstname: "Lutero"
email:
0: "lauryn.lutero@etu.univ-grenoble-alpes.fr"
followed_modules:
0: "WEB01"
birthdate: "2000-02-04"
ine: "6573953300"


http://localhost:3001/students/abcde
Import bookmarks...
Course overview
SMPE/sessions/2025_...
JSON
Raw Data
Headers
Save
Copy
Collapse All
Expand All
Filter JSON
error: "Student not found"